DAY 04 / 15 · PHASE 1 — FOUNDATIONS

Merkle trees, forks & consensus

Sunday 18 October 2026 · Manish Khadka · github.com/Khadka1996/Learning-Blockchain/tree/main/day-04

Learning objectives

1. Build a Merkle root for a list of transactions and explain why one 32-byte hash can stand for all of them.
2. Create and verify a Merkle proof, and explain why its size grows with $\log_2(n)$.
3. Know three classic Merkle-tree attacks (duplicate last node, second preimage, position lies) and the fixes.
4. Explain why forks happen, why the rule is "most total *work*", not "most blocks", and what a reorg undoes.
5. Check a payment like a phone wallet (SPV): headers only + a Merkle proof + confirmations.

The concept, step by step

Step 1 — A knockout tournament for fingerprints

Hash every transaction (a **leaf**). Pair the hashes and hash each pair. Pair the results and hash again until one hash is left: the **Merkle root**. It goes into the block header, so the block hash covers every transaction. Change one transaction and the root changes.

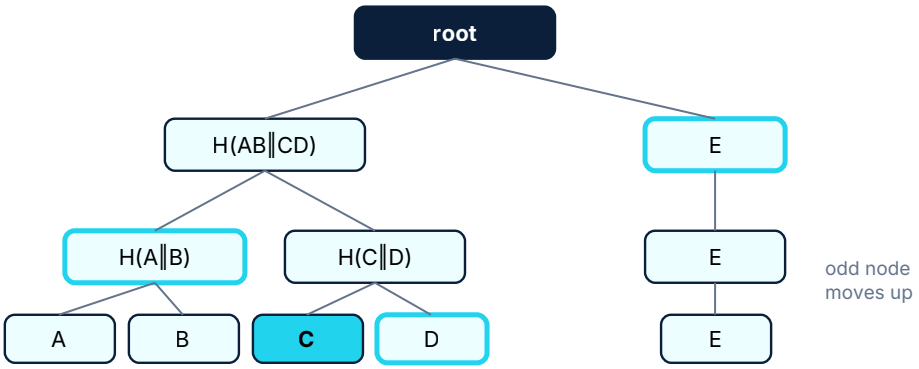


Figure 1 — 5 transactions. To prove C you need only D, H(A‖B) and E (outlined): 3 hashes, not 5 transactions.

Step 2 — A proof is your path to the final

To prove "C is in the block" you give the **siblings** C met on the way up. The verifier rebuilds the path and compares the result with the root in the header. For 1,000 transactions that is 10 hashes (320 bytes); for 1,000,000 it is 20 hashes (640 bytes). Every time the block doubles, the proof grows by one hash.

Step 3 — Tag your hashes

A naive tree can be fooled. Our rules: leaves are `H(0x00‖data)` and inner nodes `H(0x01‖left‖right)` (RFC 6962), so an inner node can never pose as a transaction; an odd node moves up unchanged instead of pairing with itself (Bitcoin's CVE-2012-2459); the root also hashes the **number** of leaves, so a proof cannot lie about the position. Tonight added a fourth rule: the leaf includes the **signature**, not only the transaction id (finding E1).

Step 4 — Forks and the most-work rule

Messages take time to cross the network. If two miners find a block at almost the same moment, half the network sees one and half sees the other: a **fork**. Each node keeps the block it saw first. When the next block lands on one branch, that branch has more total work ($\Sigma\ 2^{bits}$), and every node switches to it: a **reorg**. Payments in the losing block are undone.

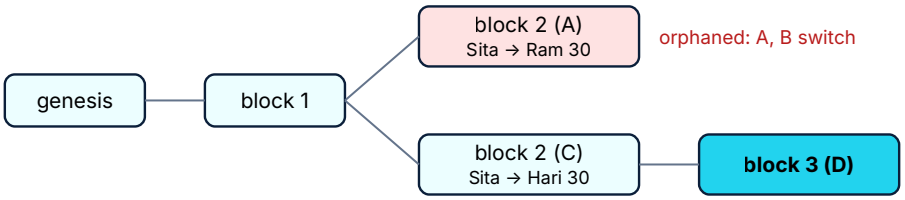


Figure 2 — the double spend from `npm run fork`: Ram's payment was "confirmed" for about 1 second, then disappeared.

Step 5 — A phone wallet (SPV)

A light client keeps only block **headers**, checks their Proof of Work, follows the header chain with the most work, and asks a full node for a Merkle proof of its payment. It learns "paid, N confirmations deep" without downloading anyone else's payments (Bitcoin whitepaper §8).

Key code walkthrough

Verifying a proof (`src/merkle.js`)

```
let h = leafHash(leaf); let i = index; let width = leafCount; let used = 0;
while (width > 1) {
  if (i % 2 === 1)        h = nodeHash(sibs[used++], h);  // we are the RIGHT child
  else if (i + 1 < width) h = nodeHash(h, sibs[used++]);  // we are the LEFT child
  /* else: last node of an odd level, moves up unchanged */
  i >>= 1; width = Math.ceil(width / 2);
}
if (used !== sibs.length) return false;                  // extra hashes = some other tree
return finalRoot(leafCount, h).toString("hex") === rootHex; // root commits to the count
```

Left/right is worked out from `index` and `leafCount` only; the proof carries nothing but hashes, so it cannot "choose" a side.

The evening fix: the leaf covers the signature (`src/block.js`)

```
function txLeaf(tx) {
  if (typeof tx.signature !== "string") throw new TypeError("signature must be a string");
  // morning: Buffer.from(txId(tx), "hex")  ← txId leaves the signature OUT
  return Buffer.from(JSON.stringify([LEAF_DOMAIN, txId(tx), tx.signature]), "utf8");
}
const txRoot = (txs) => merkleRoot(txs.map(txLeaf));
```

Fork choice and remembering invalid blocks (`src/node.js`)

```
// #connect(): fork choice by total work (workOf(bits) = 2^bits)
const work = parent.work + workOf(block.header.bits);
if (work > this.#blocks.get(this.#tip).work) this.#tip = block.hash; // tie: keep first

// receive(): a known-bad block, or a child of one, is refused at once (E2)
if (this.#invalid.has(block.hash)) return { status: "rejected", reason: "known invalid
block" };
if (this.#invalid.has(block.header.prevHash)) this.#markInvalid(block.hash);
```

Light client check (`src/spv.js`)

```
verifyPayment(tx, proof, blockHash) {
  const v = verifyTransfer(tx);                        // a real signed payment (Day 03)
  if (!this.#onBestChain(blockHash)) return { ok: false, reason: "… stale branch" };
  if (!verifyProof(txLeaf(v.tx), proof, entry.header.merkleRoot)) return { ok: false, … };
  return { ok: true, confirmations: this.tip.height - entry.height + 1 };
}
```

Results from the run

Run	Result
`npm start` — 1,000 signed payments	One root `ba173f79…`; proof for #737 = 10 hashes = 320 bytes (100× less than 32,000 bytes of ids); wrong amount → false; wrong position #738 → false
Proof size	1 → 0 hashes · 10 → 4 · 1,000 → 10 · 1,000,000 → 20 (640 bytes)
`npm run fork` — 4 nodes, double spend	Split 2–2 at t = 1.1 s; D mines block 3; A and B each throw away 1 block at t = 2.30 s; all agree; Ram 0, Hari 30
`npm run sim` — 5 miners, 300 blocks, 1 s blocks	Wasted blocks: 10 ms delay 0.3 % · 100 ms 6.0 % · 300 ms 16.0 % · 1,000 ms 37.7 % (deepest reorg 4)
`npm run spv` — 30 blocks, 600 payments	Full blocks 205,611 bytes vs headers 5,986 bytes (34× less); Ram's payment: 5-hash proof, 26 confirmations; 250-coin lie, stale-branch payment and no-work header all refused
Quality	44 / 44 tests pass · ESLint clean · `npm audit` 0 vulnerabilities

QA & security findings

Morning: 8 findings fixed (1 high, 5 medium, 2 low). Evening audit as an outsider: 3 more (1 medium, 2 low), each with a failing test first.

ID	Sev.	Problem	Fix
A3	**High**	"Longest chain" counted blocks and trusted each block's difficulty: 4 blocks worth ~8 guesses beat 2 honest blocks worth ~131,000.	Most total work; difficulty is a network rule.
A1, A2, M1	**Medium**	Duplicate-last trick, inner node passed off as a leaf, proof lying about the position.	Odd node moves up; 0x00/0x01 tags; root commits to the leaf count.
A4, M2	**Medium**	Unlimited orphan pool; a getter-based payment changing after acceptance bricked the node.	PoW-checked, capped pool; frozen copies of every transaction.
E1	**Medium**	Leaves used `txId`, which excludes the signature: a copy with a broken signature had the same block hash and, sent first as an orphan, made the node drop the real block as a "duplicate" (victim stuck at height 1, Ram 0 instead of 30).	Leaf = [txId, signature]: the block hash covers every signature.
E2	**Low**	Invalid blocks were forgotten: 20 resends at height 100 cost 2,044 ms of replays.	Remember permanently invalid blocks and their children (86 ms). Temporary failures are not remembered.
E3	**Low**	`chain()` / `ledgerAt()` crashed on an unknown hash.	Clear `RangeError`.

Known issues kept: ledger replay from genesis per block (K1), orphan / invalid-set eviction by paid PoW (K2), no mempool or deep-reorg limit (K3), 2-hour future timestamps (K4), no byte limit per block (K5), and light clients trust that full nodes validated payments (K6).

Common mistakes

- Pairing the last odd node with itself — two different lists get the same root.
- Hashing leaves and inner nodes the same way — a 64-byte node becomes a fake "transaction".
- Letting the proof say left/right — work it out from index and leaf count.
- Committing to data that leaves something out (here: the signature). Ask "what exactly does this hash cover?"
- Following the chain with the most blocks instead of the most work.
- Treating 1 confirmation as final. Forks are normal; finality is a probability.

Interview questions

1. Why is a Merkle proof O(log n)?

Each level halves the number of nodes, and the proof needs one sibling per level: $\lceil \log_2 n \rceil$ hashes. 1,000 leaves → 10 hashes.

2. What is CVE-2012-2459?

Bitcoin pairs an odd last node with itself, so [a,b,c] and [a,b,c,c] share a root. A node could be fed an invalid "mutated" version of a valid block with the same hash. Fix: move the odd node up (or reject duplicates) and never cache "invalid" for a hash that has a valid twin.

3. Why do Merkle trees use different prefixes for leaves and nodes?

To prevent second-preimage attacks: without them, the 64 bytes of an inner node can be presented as a leaf and verify against the same root (RFC 6962 §2.1).

4. "Longest chain" — longest by what?

By total work (sum of expected hashes, 2^{bits} per block). Counting blocks would let an attacker win with many cheap blocks.

5. What does an SPV client trust?

That the chain with the most work contains only valid transactions. It checks PoW and inclusion, not validity, so it is safe only while honest miners hold most of the hash power.

Exercises summary

Level	Task	Result of the solution
Easy	`proofLength(n, index)` by arithmetic only	Matches `merkleProof` on all 2,080 (size, index) pairs; in a 5-leaf tree leaf 4 needs only 1 hash
Medium	`findDifferences(a, b)`, top-down	1 changed payment of 1,024 found with 21 comparisons; 4 changed with 69
Challenge	Multiproof for many leaves	8 of 1,000 payments: 27 hashes instead of 78; tampering → false

Cheat sheet

leaf	`H(0x00 ‖ JSON[tx-leaf, txId, signature])`
node	`H(0x01 ‖ left ‖ right)`; odd node moves up unchanged
root	`H(0x02 ‖ count(8 bytes) ‖ top)` — in the block header
proof size	$\lceil \log_2 n \rceil \times 32$ bytes (1,000 → 320 B; 1,000,000 → 640 B)
work	2^{bits} per block; best chain = most total work; tie → first seen
reorg	switch branch; blocks above the fork point are undone
SPV	headers + Merkle proof + confirmations = tip height − block height + 1
commands	`npm start · fork · sim · spv · attack:evening · ex:easy|medium|challenge · test`

Key terms

Term	Plain meaning
Merkle root	One hash in the block header that stands for every transaction in the block
Merkle proof	The sibling hashes on the path from one leaf to the root (about $\log_2 n$ of them)
Multiproof	One proof for several leaves; shared hashes are sent once (exercise 3: 27 instead of 78)
Fork	Two valid blocks with the same parent, usually because news travels at a finite speed
Reorg	A node switches to a branch with more work; blocks on the old branch are undone
Orphan (this code)	A block whose parent the node has not seen yet; kept (max 100) and connected later
Stale block	A valid block that ended up off the winning chain: wasted work
Work	Expected guesses to find a block, 2^{bits}; a chain's work is the sum
SPV / light client	Checks payments with headers + Merkle proofs, without every block
Confirmations	How many blocks, counting its own, sit on top of a payment's block

What I'd explore next

- Bitcoin's real 80-byte header, double SHA-256 and the compact `nBits` target; verify a real block's Merkle root.
- Merkle Patricia tries and Verkle trees: Ethereum commits to every balance, so a light client can prove "Sita has 70 coins".
- Selfish mining (Eyal & Sirer, 2014): simulate a miner that hides blocks with today's network code.

References

1. S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System", §7–8 — https://bitcoin.org/bitcoin.pdf
2. R. Merkle, "A Digital Signature Based on a Conventional Encryption Function", CRYPTO '87 — https://link.springer.com/chapter/10.1007/3-540-48184-2_32
3. RFC 6962, Certificate Transparency, §2.1 Merkle Hash Trees — https://www.rfc-editor.org/rfc/rfc6962#section-2.1
4. CVE-2012-2459 (duplicate transactions in the Merkle tree) — https://nvd.nist.gov/vuln/detail/CVE-2012-2459
5. BIP 141, Segregated Witness: witness commitment — https://github.com/bitcoin/bips/blob/master/bip-0141.mediawiki
6. Bitcoin Developer Guide: Block Chain — https://developer.bitcoin.org/devguide/block_chain.html
7. A. Antonopoulos & D. Harding, Mastering Bitcoin, 3rd ed., ch. 11–12 — https://github.com/bitcoinbook/bitcoinbook
8. ethereum.org: Merkle Patricia Trie — https://ethereum.org/en/developers/docs/data-structures-and-encoding/patricia-merkle-trie/
9. I. Eyal & E. G. Sirer, "Majority is not Enough: Bitcoin Mining is Vulnerable" (2014) — https://arxiv.org/abs/1311.0243